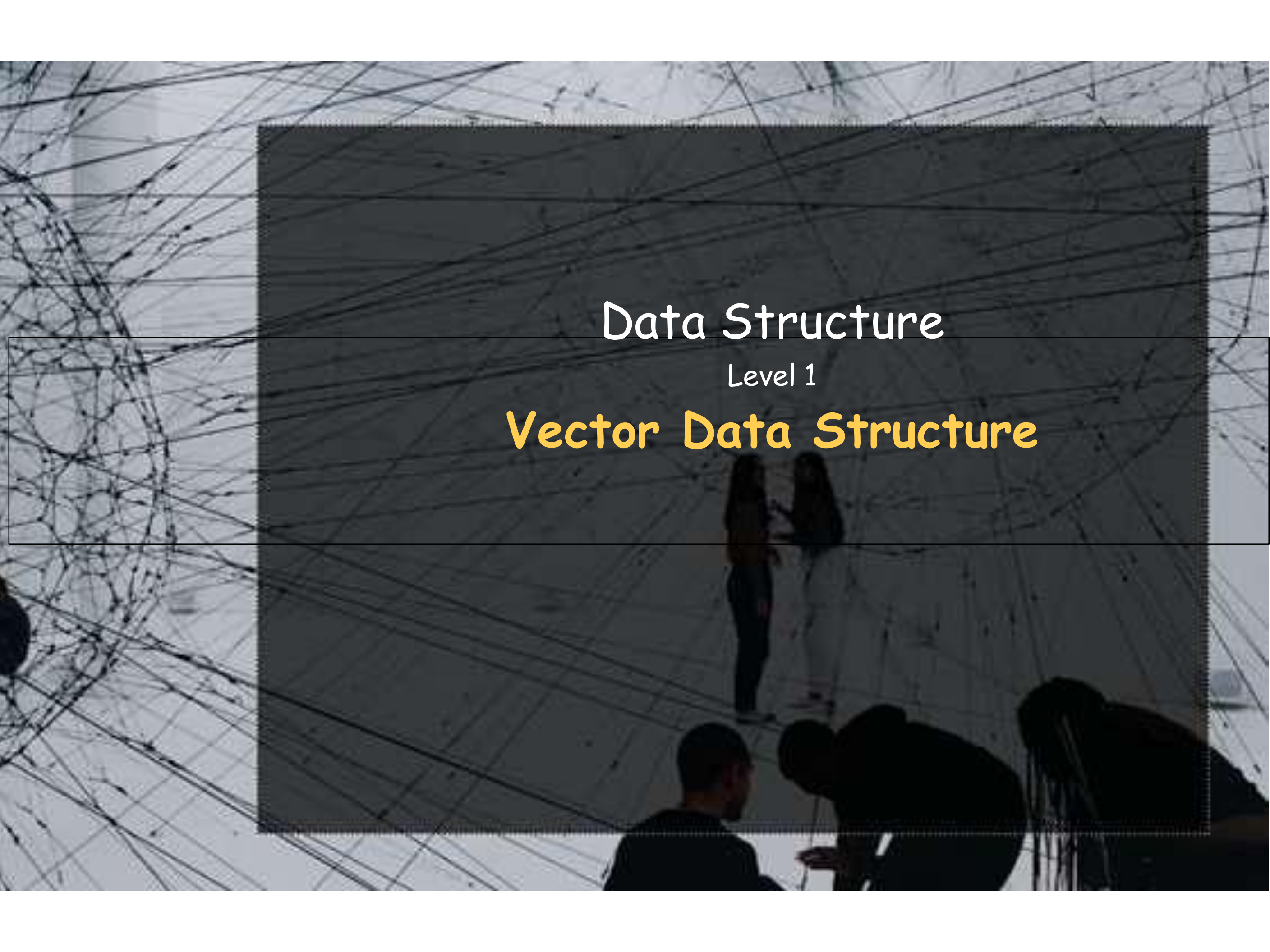


Data Structure Level 1

Vector Data Structures

The background of the slide features a complex, abstract network of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, some standing and some appearing to be in a discussion or collaborative work, adding a human element to the technical theme.

Data Structure

Level 1

Vector Data Structure

Definition

A Vector is a sequence container that stores elements in contiguous memory and allows dynamic resizing.

Explanation

- ◆ Vectors work like arrays but are more powerful.
- ◆ They automatically handle memory allocation.
- ◆ Elements can be accessed using an index like arrays.
- ◆ They support many built-in functions for easy data manipulation

Common Vector Operations

-  `push_back(x)` → Add element at the end
-  `pop_back()` → Remove last element
-  `size()` → Returns number of elements
-  `empty()` → Checks if vector is empty
-  `at(index)` → Access element safely
-  `clear()` → Remove all elements

Procedures (Program Flow)

- 1 Include vector library → `#include <vector>`
- 2 Create vector → `vector<int> v;`
- 3 Add elements → `v.push_back(10);`
- 4 Access elements → `v[0]` or `v.at(0)`
- 5 Get size → `v.size()`
- 6 Remove last element → `v.pop_back()`

Example

If vector contains:
10, 20, 30

After push_back(40)

➡ Result:

10, 20, 30, 40



Comparison

■ Vector

- ✓ Dynamic size
- ✓ Automatic memory management
- ✓ Many built-in functions
- ✓ Easy insertion at end

■ Array

- ✓ Fixed size
- ✓ Manual memory handling
- ✓ Fewer built-in utilities
- ✓ Faster for fixed data

Note

Vectors store elements in continuous memory, which makes access very fast.

Summary

- ✓ Vector is a dynamic array
- ✓ Supports automatic resizing
- ✓ Provides many useful built-in functions
- ✓ Widely used in modern C++ programming

Thank you 😊